



มหาวิทยาลัยธรรมศาสตร์
THAMMASAT UNIVERSITY

การพัฒนาระบบเฝ้าระวังและบริหาร จัดการชุมชน (Sentinel Guard)

โครงการอบรมสำหรับนักศึกษาหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาศาสตร์และนวัตกรรมข้อมูล
วิทยาลัยสหวิทยาการ มหาวิทยาลัยธรรมศาสตร์



เอกสารประกอบการอบรม



[Google Slide](#)

วิทยากรบรรยาย



- ชื่อ : นางสาวสุภัทสรณ์ สุขภูติ
- หน่วยงาน: บริษัท เคซี ออล จำกัด
- ประวัติ:
 - ปริญญาตรี คณะวิทยาศาสตร์และเทคโนโลยี สาขา วิทยาการคอมพิวเตอร์
 - ตำแหน่ง Full-Stack Developer
- ช่องทางการติดต่อ:
 - praewsps1334@gmail.com

Scan the QR code to join our LINE group



Workshop Agentic Living Lab 2026

กำหนดการอบรม

- **วันที่:** วันที่ 21 มีนาคม 2569
เวลา: 08:30 - 16:00 น.
- **สถานที่:** INC1-101 และทาง Online

เวลา	กิจกรรม
วันที่ 21 มีนาคม 2569	การพัฒนาระบบเฝ้าระวังและบริหารจัดการชุมชน (Sentinel Guard)
08:30 - 09:00	ลงทะเบียนและรับเอกสารประกอบการอบรม
09:00 - 10:30	บรรยายทฤษฎีรูปแบบของ Software Logging และหลักการ Automated QA
10:30 - 10:45	พักเบรก
10:45 - 12:00	ปฏิบัติการช่วงที่ 1: การสร้างตรรกะวิเคราะห์ Error Log
12:00 - 13:00	พักรับประทานอาหารกลางวัน
13:00 - 14:30	ปฏิบัติการช่วงที่ 2: การพัฒนา Chatbot เบื้องต้นสำหรับการรับแจ้งปัญหา
14:30 - 14:45	พักเบรก
14:45 - 16:00	การบูรณาการระบบ QA เข้ากับ Workflow การทำงาน สรุปถาม-ตอบก่อนปิดการอบรม

หลักการและเหตุผล

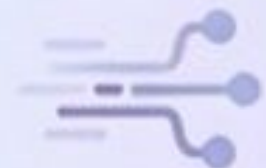
การดูแลรักษาระบบที่มีผู้ใช้งานจำนวนมาก และการบริหารจัดการบริการชุมชนที่มีความหลากหลาย จำเป็นต้องมีเครื่องมือที่ช่วยลดภาระงานของมนุษย์ การใช้ระบบอัตโนมัติในการวิเคราะห์ข้อผิดพลาด (Log Analysis) และการโต้ตอบกับผู้ใช้งานเบื้องต้น (Chatbot) จะช่วยเพิ่มประสิทธิภาพในการให้บริการและแก้ไขปัญหาได้อย่างรวดเร็ว

ดังนั้น โครงการนี้จึงมุ่งเน้นการพัฒนาระบบเฝ้าระวังอัจฉริยะ (Intelligent Sentinel) ที่สามารถวิเคราะห์หาสาเหตุของปัญหาทางเทคนิคได้โดยอัตโนมัติ และระบบบริหารจัดการบริการสำหรับชุมชนที่ใช้งานง่ายผ่านโทรศัพท์มือถือ ซึ่งสอดคล้องกับแนวทางการสร้างเมืองอัจฉริยะ (Smart City) และการนำเทคโนโลยีมาใช้เพื่อยกระดับคุณภาพชีวิตของคนในชุมชน

วัตถุประสงค์โครงการ

1. เพื่อพัฒนาระบบวิเคราะห์รากฐานของปัญหา (Root Cause Analysis) จาก Log Files โดยอัตโนมัติ
2. เพื่อสร้างระบบแคตตาล็อกบริการ (Service Catalog) และ Chatbot สำหรับร้านค้าชุมชน
3. เพื่อถ่ายทอดความรู้ด้าน Automated QA และ Chatbot Development

รูปแบบของ Software Logging และหลักการ Automated QA



The "Black Box" Problem

ซอฟต์แวร์ที่ไม่มี Log คือ **กล่องดำ**

เวลาที่แอปพลิเคชันไปรันบน Production เราไม่สามารถนั่งดูมันทำงานที่ละบรรทัดได้เหมือนตอนอยู่บนเครื่อง Dev

 User: "พี่คะ กดโอนเงินแล้วมันค้างค่ะ"

 Dev: "อ้าว... ในเครื่องผมรันได้ปกตินะ?"

ถ้าไม่มี Log เราก็ทำได้แค่เดาว่าเกิดอะไรขึ้น...

3 เสาหลักของประโยชน์จาก Log



Debugging

ช่วยหา Root Cause ว่าระบบพัง
ที่ไหน พังเพราะอะไร เพื่อแก้ไข



Monitoring

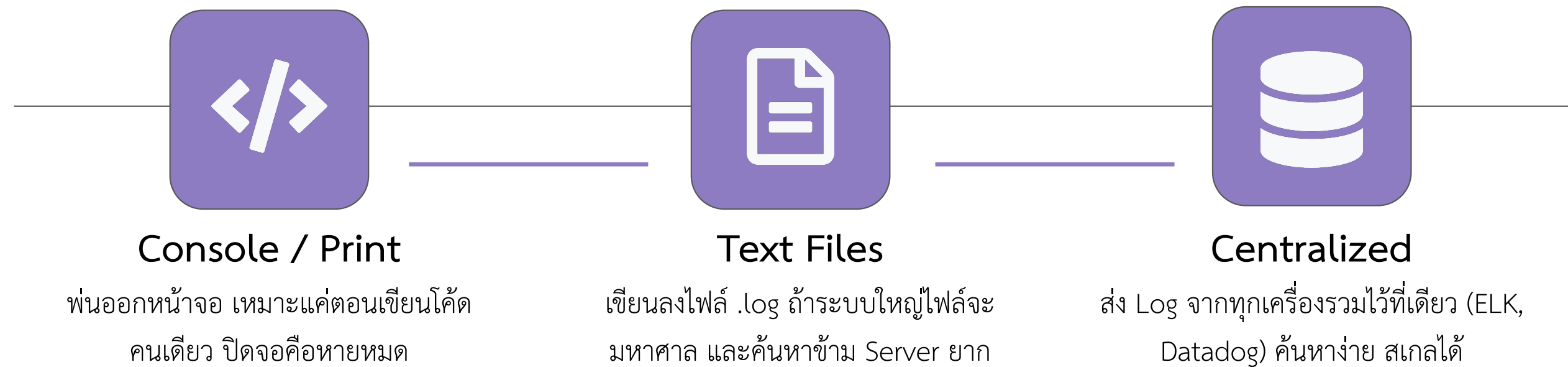
สังเกตการณ์ว่าระบบยังทำงาน
ปกติไหม (Health Check)



Auditing & Security

ติดตามร่องรอยว่า "ใคร" ทำ
"อะไร" "เมื่อไหร่" และ "จากที่ไหน"

วิวัฒนาการของการเก็บ Log



Bad Log vs Good Log

❌ Bad Log: ขาดข้อมูลระบุตัวตนและเหตุการณ์แวดล้อม

```
Error: Cannot find user  
User login failed  
console.log("here 1")
```

✅ Good Log: ตอบคำถาม When, Who, What, Where, Why ได้ครบจบในบรรทัดเดียว

```
[ERROR] 2026-03-18T12:55:19Z  
User_ID: 1042  
Action: Login  
Reason: Invalid Password  
IP: 192.168.1.1
```

Logging Levels

FATAL / CRITICAL

ความหมาย: ระบบล่มพังทลาย (System Crash) แอปไม่สามารถทำงานต่อได้เลย

Action ที่ต้องทำ: ปลุก Dev/DevOps ลุกขึ้นมาแก้เดี๋ยวนี้

ตัวอย่าง: Database connection lost, Out of Memory

ERROR

ความหมาย: เกิดข้อผิดพลาดในบางฟังก์ชัน (แต่ระบบโดยรวมยังเดินต่อได้)

Action ที่ต้องทำ: สร้าง Ticket ให้ทีมรีบตรวจสอบในเวลาทำการ

ตัวอย่าง: ผู้ใช้โอนเงินไม่สำเร็จเพราะ API Gateway
ปลายทางล่ม, จ่ายเงินผ่านบัตรเครดิตพัง

Logging Levels

WARNING (WARN)

ความหมาย: สิ่งผิดปกติที่เกิดขึ้น "ยังไม่ใช่ปัญหาในตอนนี้" แต่อาจจะบานปลายกลายเป็น ERROR ได้ในอนาคตอันใกล้

ตัวอย่าง: Disk space usage ถึง 85% แล้ว, API เส้นนี้ตอบสนองช้ากว่า 5 วินาที (เสี่ยง Timeout), มีการเรียกใช้ฟังก์ชันเก่าที่กำลังจะถูกยกเลิก (Deprecated API)

Logging Levels

INFO

ความหมาย: ข้อมูลทั่วไป บันทึกเส้นทางว่าระบบกำลังทำอะไรอยู่ (Happy Path)

ตัวอย่าง: `Service started on port 8000`, `User 123 logged in`, `Scheduled job finished successfully`

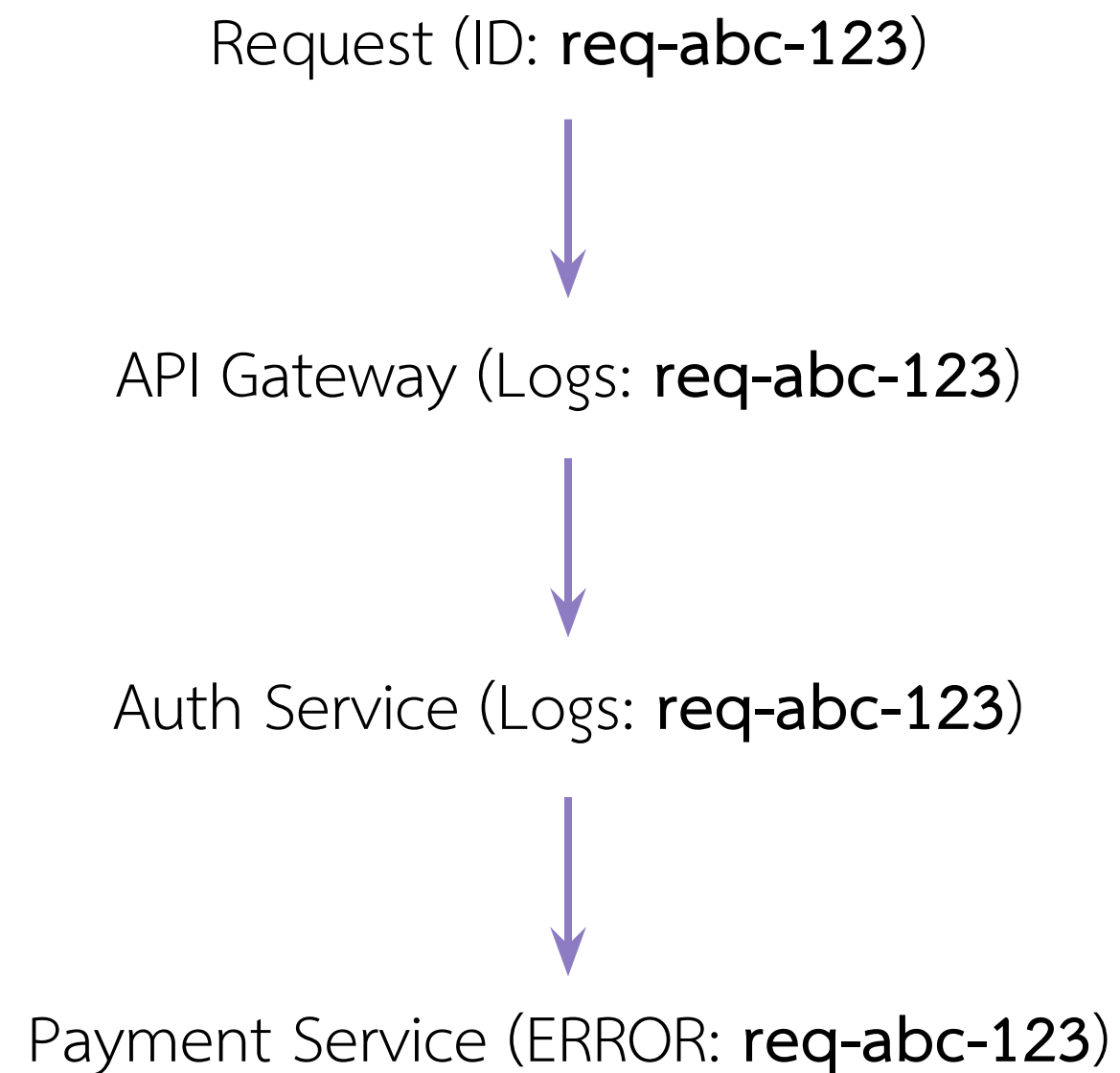
DEBUG / TRACE

ความหมาย: ข้อมูลเชิงลึกสุดๆ ข้อมูลดิบ (Payload) ใช้เฉพาะตอนหาบั๊ก

คำเตือน: ปกติจะปิดไว้บน Production เพราะจะกินพื้นที่ Disk มหาศาล และอาจมีข้อมูลรกเกินไป

Context is King: Correlation ID

เมื่อทำระบบ Microservices เราจะได้รู้ว่า Log จาก 3 เซิร์ฟเวอร์ เป็นของ User คนเดียวกัน?



เราใช้ `Correlation ID` เป็นกุญแจในการค้นหา Log ตลอดทั้งเส้นทาง!

ข้อห้าม

"NEVER LOG PII (Personally Identifiable Information)"

ห้าม Log รหัสผ่าน, หมายเลขบัตรเครดิต, หรือข้อมูลส่วนบุคคลที่ละเอียดอ่อนลงไปเด็ดขาด (ผิดกฎหมาย PDPA/GDPR เต็มๆ)

* ต้องทำการ Masking/Hash ข้อมูลก่อนเสมอ เช่น

****_****_****-1234

Structured Logging (JSON)

ปัญหาของ Plain Text คือเวลาเอา Log ไปค้นหาในระบบใหญ่ๆ
ทำได้ยากมาก ต้องเขียน Regex ให้ปวดหัว

ทางแก้: ใช้รูปแบบ JSON Format

☰ ระบบวิเคราะห์ (Elasticsearch, Datadog) สามารถจับคู่
Key-Value ได้ทันที

📊 นำไปทำ Dashboard หรือตั้ง Alert ได้ง่ายดาย

🔍 Filter หา `userId: 123` ได้ในเสี้ยววินาที



Plain Text vs Structured Logging (JSON)

Plain Text

2026-03-18 10:00:00 [ERROR] User 123 failed
login IP 1.1.1.1

Structured JSON

```
{  
  "timestamp": "2026-03-18T10:00:00Z",  
  "level": "ERROR",  
  "event": "login_failed",  
  "userId": "123",  
  "ip": "1.1.1.1"  
}
```

สถาปัตยกรรม Logging ของ Python / Django

1 Loggers

จุดรับข้อมูล (Entry point) ในโค้ดของเรา เพื่อตั้งระดับความรุนแรง



2 Filters

ตัวกรองว่าจะให้ Log บรรทัดนี้ผ่านไปได้หรือไม่ (เช่น ให้ผ่านเฉพาะตอน DEBUG=True)



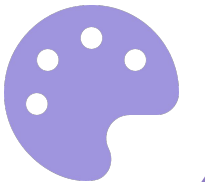
3 Handlers

ตัวกำหนดทิศทางว่าจะส่ง Log ไปที่ไหน (ออก Console, ลงไฟล์, ส่งเมล)



4 Formatters

ตัวแปลงหน้าตาของข้อความ (เช่น จาก Text ธรรมดา เปลี่ยนเป็น JSON)



Loggers (จุดรับข้อมูล)

เรามักจะประกาศตัวแปร `logger` ไว้บนสุดของไฟล์ และตั้งชื่อตามชื่อ Module (ไฟล์) เพื่อให้รู้ว่า Log นี้มาจากไฟล์ไหน

- ✓ `__name__` จะได้ชื่อไฟล์อัตโนมัติ เช่น `myapp.views`
- ✓ Logger จะตรวจสอบ Level ของตัวเอง ถ้าต่ำกว่าที่ตั้งค่าไว้ มันจะไม่ส่งต่อให้ Handler

```
import logging

# สร้าง Logger ประจำไฟล์นี้
logger = logging.getLogger(__name__)

def my_function():
    # เรียกใช้งาน
    logger.debug("Start processing...")
    logger.info("User clicked button.")
    logger.warning("Disk space low.")
    logger.error("Failed to save to DB.")
    logger.critical("System is down!")
```

Handlers (กำหนดปลายทาง)

Log 1 ตัว สามารถผูกได้หลาย Handler เช่น พิมพ์ออกจอต้วย และเขียนลงไฟล์ด้วยพร้อมกัน

➤ **StreamHandler:** ผนอก Console/Terminal (เหมาะกับ Docker/Kubernetes)

📄 **FileHandler / RotatingFileHandler:** เขียนลงไฟล์ (และตัดไฟล์ใหม่เมื่อขนาดใหญ่เกิน)

✉ **AdminEmailHandler:** ของแถมจาก Django ถ้ามี ERROR จะส่ง Email แจ้ง Admin ทันที

Filters & Formatters

Filters (ตัวกรอง)

ควบคุมลอจิกพิเศษ เช่น `RequireDebugFalse` ของ Django จะยอมให้ Log ระดับ ERROR ทำงาน (ส่งอีเมลหา Admin) ก็ต่อเมื่อเราตั้งค่า `DEBUG = False` (อยู่บน Production) เท่านั้น

Formatters (รูปแบบหน้าต่างา)

เราสามารถใช้ตัวแปรพิเศษที่ Python เตรียมไว้ให้จัดหน้าต่างา Log ได้ เช่น

`%(asctime)s` = เวลาปัจจุบัน

`%(levelname)s` = INFO, ERROR

`%(name)s` = ชื่อ Logger (ไฟล์)

`%(message)s` = ข้อความที่เราส่งเข้าไป

Django Config (JSON Log)

เปลี่ยนให้ Django พ่น Log ออกมาเป็น JSON ด้วยไลบรารี

python-json-logger

เราแค่เพิ่ม Formatter ใหม่เข้าไป และผูกเข้ากับ Handler ที่เราใช้งาน (เช่น Console หรือ File)

* อย่าลืม `pip install python-json-logger`

```
# settings.py
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json_formatter': {
            '():': 'pythonjsonlogger.jsonlogger.JsonFormatter', # ใช้ JSON
            'format': '%(asctime)s %(levelname)s %(name)s %(message)s'
        },
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'json_formatter', # ผูก Formatter เข้ากับ Handler
        },
    },
    'loggers': {
        'myapp': { # ชื่อแอปของเรา
            'handlers': ['console'],
            'level': 'INFO', # เก็บตั้งแต่ INFO ขึ้นไป
        },
    },
}
```

Using Logger in Django Views

บอกลา `print()`

เวลาเขียนโค้ดใน Views ห้ามใช้ print เต็ดขาด! ให้ใช้ `logger` แทน และส่งตัวแปร **extra={}** เข้าไปด้วยเสมอเพื่อบอก "บริบท" ว่าใครกำลังทำอะไร

✓ เก็บ Error Message คู่กับ Context

✓ ไม่ต้องต่อ String เองให้วุ่นวาย

```
# views.py
import logging
logger = logging.getLogger('myapp')

def checkout_view(request):
    user_id = request.user.id

    try:
        # Good Log: แจ้งสำเร็จพร้อม Context
        logger.info(
            "Checkout successful",
            extra={'user_id': user_id, 'action': 'checkout'}
        )
        return render(request, 'success.html')

    except Exception as e:
        # Good Log: เก็บสาเหตุการพัง
        logger.error(
            "Checkout failed gateway error",
            extra={'user_id': user_id, 'err': str(e)}
```


Logging Middleware

ใน Django เราสามารถเขียน Middleware เพื่อดักจับทุกๆ HTTP Request และ Response ที่วิ่งเข้าออกระบบได้

- ✓ บันทึก Method (GET/POST) และ URL อัตโนมัติ
- ✓ คำนวณเวลาที่ใช้ประมวลผล (Response Time)
- ✓ เก็บ IP Address ของผู้ใช้

```
class RequestLoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # จับเวลาก่อนเริ่มทำงาน
        start_time = time.time()

        # ปล่อยให้ View ทำงาน
        response = self.get_response(request)

        # คำนวณเวลาและ Log
        duration = time.time() - start_time
        logger.info("HTTP Request", extra={
            'method': request.method,
            'path': request.path,
            'status': response.status_code,
            'duration_s': duration,
            'ip': request.META.get('REMOTE_ADDR')
        })
        return response
```

Centralized Logging Tools

เมื่อ Log เป็น JSON แล้ว เราจะส่งไปเก็บที่ไหน?

ELK Stack (Open Source / Self-Hosted)

Elasticsearch (Database เก็บข้อมูล ค้นหาเร็วมาก), Logstash (ตัวส่งข้อมูล), Kibana (หน้า Dashboard)

Cloud-Native / SaaS

Datadog, New Relic, Sentry, AWS CloudWatch, GCP Cloud Logging

Automated QA คืออะไร?

การเปลี่ยนจากการคลิกทดสอบระบบด้วยมือ (Manual) มาเป็นการเขียน "สคริปต์" ให้คอมพิวเตอร์ทดสอบแทน



Fast & Repeatable

รันกี่พันครั้งก็ได้ เร็วและไม่เหนื่อย



Prevent Regression

มั่นใจว่าฟีเจอร์ใหม่จะไม่ทำ
ของเก่าพัง



Documentation

เทสต์ไค์ดคือเอกสารอธิบายการทำ
งานที่อัปเดตที่สุด

Manual vs Automated Testing

Manual QA (ทดสอบด้วยมือ)

คนนั่งคลิกหน้าเว็บ กรอกฟอร์มที่ละช่อง

ข้อดี: ตรวจสอบความสวยงาม (UI/UX) ได้ดี

ข้อเสีย: ช้า, น่าเบื่อ, คนอาจตาตายและพลาดได้ (Human Error) เมื่อระบบใหญ่ขึ้น ไม่สามารถคลิกทดสอบ "ทุกฟีเจอร์" ได้ทุกวัน (Regression เป็นไปได้ยาก)

Automated QA (โค้ดทดสอบ)

เขียนสคริปต์ให้คอมพิวเตอร์ทดสอบแทนคน

ข้อดี: เร็วมาก! รันกี่พันครั้งก็ได้ผลเหมือนเดิม, ป้องกัน Regression (ฟีเจอร์เก่าพังเพราะโค้ดใหม่) ได้ 100%

ข้อเสีย: มีต้นทุนในการเรียนรู้และเขียนสคริปต์เทสต์ครั้งแรก

F.I.R.S.T Principles

หลักการเขียน Automated Test ที่ดี (อย่าปล่อยให้เกิด Flaky Tests)

F - Fast: ต้องทำงานเร็ว เพื่อให้ Dev กล้ารันบ่อยๆ ทุกครั้งที่เซฟโค้ด

I - Independent: เทสต์แต่ละข้อต้องไม่พึ่งพากัน รันข้อไหนก่อน-หลัง เดียวๆ ก็ต้องผ่าน

R - Repeatable: รันกี่ครั้ง ในสภาพแวดล้อมไหน ผลก็ต้องเหมือนเดิมเสมอ

S - Self-Validating: สคริปต์ต้องสรุปผลตัวเอง (Pass/Fail) ไม่ต้องใช้คนมานั่งเพ่ง Log

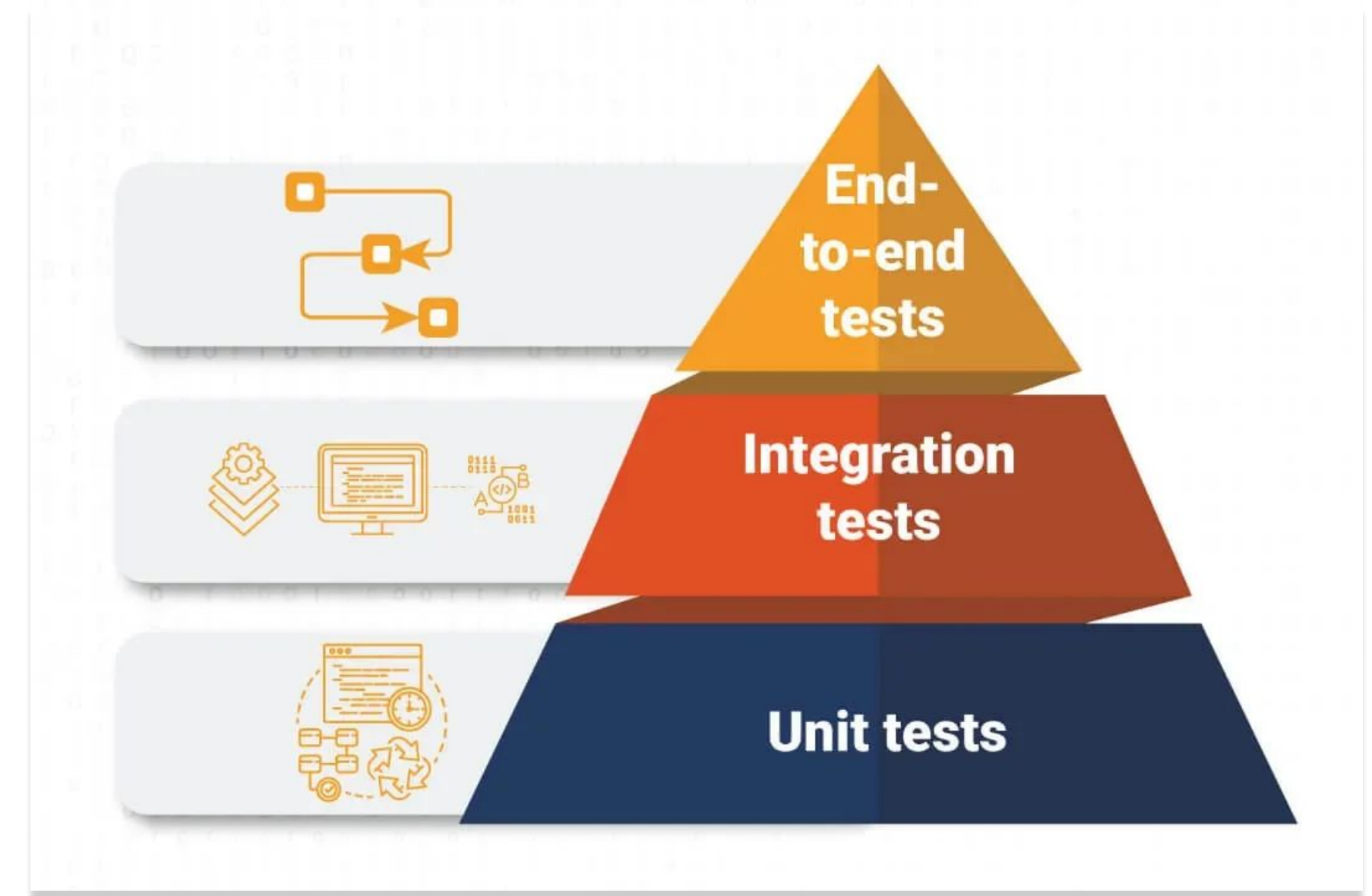
T - Timely: ควรเขียนเทสต์ควบคู่ไปกับการเขียนโค้ด (หรือทำ TDD - Test Driven Development)

The Test Automation Pyramid

Unit Tests (ฐาน - ควรมีเยอะสุด): ทดสอบระดับฟังก์ชัน
เล็กๆ รันเร็วมาก ต้นทุนถูก (เช่น Jest, PyTest)

Integration Tests (ตรงกลาง): ทดสอบการทำงานร่วมกัน
เช่น API คู่กับ Database (เช่น Supertest)

E2E Tests (ยอด - มีแค่ Core Flow): จำลองพฤติกรรม
User คลิกปุ่มจริง (รันช้า พังง่าย ต้นทุนแพง เช่น Cypress)



Django Test Superpowers

Isolated Test Database

ความเทพของ Django คือเวลาสั่งรัน `python manage.py test` ระบบจะสร้าง **Test Database** ขึ้นมาใหม่แยกจากของจริงให้เลย ข้อมูลเทสต์จะไม่ปนกับของ Production อย่างเด็ดขาด

Automatic Rollback

เมื่อรันเทสต์เสร็จแต่ละข้อ (แต่ละฟังก์ชัน) Django จะ **Rollback (ล้างข้อมูล)** ให้กลับมาเป็นเหมือนเดิมเป๊ะ! แปลว่าเทสต์แต่ละข้อจะ Independent 100% ข้อมูลจะไม่กวนกัน

Anatomy of a Unit Test

แพทเทิร์น A-A-A

การทดสอบระดับ Model คือลอจิกพื้นฐานที่สุด รันเร็วที่สุด

1. **Arrange** (เตรียมการ): ตั้งค่าข้อมูลเบื้องต้นให้พร้อม
2. **Act** (ลงมือทำ): เรียกใช้งาน Method ที่ต้องการทดสอบ
3. **Assert** (ตรวจสอบ): เช็คว่าผลลัพธ์ตรงกับที่คาดหวังหรือไม่

```
# tests.py
from django.test import TestCase
from .models import Product

class ProductTest(TestCase):
    def setUp(self):
        # 1. Arrange: เตรียมข้อมูลใน Test DB
        self.product = Product.objects.create(
            name="Book", price=1000
        )

    def test_calc_tax(self):
        # 2. Act: เรียกใช้ Method (Vat 7%)
        tax = self.product.calc_tax(0.07)

        # 3. Assert: ภาษี 1000 ควรเท่ากับ 70
        self.assertEqual(tax, 70)
```

Integration Test

ทดสอบระดับ Views / API

ขยับขึ้นมาอีกนิด เราจะใช้ **self.client** ซึ่งเป็นเสมือน

เบราว์เซอร์จำลองของ Django ยิง Request ไปที่ URL เพื่อดูว่า

View ทำงานถูกไหม หน้าเว็บ หรือ JSON Response ออกมาตรงตามที่คาดหวังหรือเปล่า

```
# tests.py
from django.test import TestCase
from django.urls import reverse

class ProductViewTest(TestCase):
    def setUp(self):
        Product.objects.create(name="Book")

    def test_product_list_view(self):
        # 1. Act: จำลองการยิง HTTP GET
        res = self.client.get(reverse('product'))

        # 2. Assert: ตรวจสอบ HTTP 200 OK
        self.assertEqual(res.status_code, 200)

        # 3. Assert: ตรวจสอบคอนเทนต์ใน HTML
        self.assertContains(res, "Book")
```

Mocking: การสร้างของปลอม (Fake it!)

สมมติว่าโค้ดเรามีการเรียก API ไปตัดบัตรเครดิตของ Omise หรือ Stripe เวลาเรารันเทสต์ เราคงไม่อยากเสียเงินตัดบัตรจริงๆ ทุกครั้งที่กดรัน!

ใช้ `unittest.mock.patch` เพื่อสกัดกั้นการทำงานของฟังก์ชันนั้น

สั่งให้มันเสแสร้งว่า "ตอบกลับว่าสำเร็จแล้ว (Return True)" แทนที่โดยไม่ต้องยิงเน็ตจริง

```
from unittest.mock import patch

class PaymentTest(TestCase):

    # สกัดกั้นฟังก์ชัน call_stripe_api
    # แล้วแทนที่ด้วย mock_stripe
    @patch('myapp.services.call_stripe_api')
    def test_checkout(self, mock_stripe):
        # สั่งให้ของปลอม ตอบกลับว่า True เสมอ
        mock_stripe.return_value = True
        # Act: เรียกฟังก์ชันของเรา (ซึ่งมันจะไปเรียกของปลอมแทน)
        res = self.client.post('/checkout/')

        # Assert
        self.assertEqual(res.status_code, 200)
        # เช็คจำนวนที่ของปลอมถูกเรียกใช้งานจริงๆ 1 ครั้ง
        mock_stripe.assert_called_once()
```


Code Coverage

เครื่องมือที่จะบอกว่า Test ของเราวิ่งผ่านโค้ดที่เราเขียนไปกี่เปอร์เซ็นต์ (บรรทัดไหนโดนทดสอบแล้ว บรรทัดไหนยังไม่เคยมีสคริปต์ไปแตะเลย)

coverage run manage.py test

coverage report -m (ดูรายงาน)

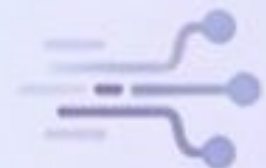
* คำเตือน: 100% Coverage ไม่ได้แปลว่าไร้บั๊ก! มันแค่บอกว่าโค้ดถูกรันผ่าน แต่ลอจิกอาจจะผิดอยู่ดี

Coverage report: 100%

Module	statements ↑	missing	excluded	coverage
conftest.py	0	0	0	100%
dashboard/admin.py	1	0	0	100%
dashboard/models.py	1	0	0	100%
users/admin.py	1	0	0	100%
cloudstack/urls.py	3	0	0	100%
dashboard/apps.py	3	0	0	100%
users/apps.py	3	0	0	100%
dashboard/urls.py	4	0	0	100%
dashboard/views.py	4	0	0	100%
users/urls.py	6	0	0	100%
users/models.py	12	0	0	100%
users/views.py	15	0	0	100%
users/forms.py	19	0	0	100%
users/managers.py	20	0	0	100%
cloudstack/settings.py	23	0	0	100%
Total	115	0	0	100%

WORKSHOP 01

การสร้างตรรกะวิเคราะห์ Error Log



Git

<https://github.com/wasit7/Living-Lab.git>

```
git clone --depth 1 --filter=blob:none --sparse https://github.com/wasit7/Living-Lab.git && cd Living-Lab && git  
sparse-checkout set 4_sentinel_guard
```

The Phantom Transfer Bug

ระบบ Wallet ของแอปเรามีฟีเจอร์ให้ User โอนเงินหากันได้ ล่าสุดมีลูกค้าโวยวายมาว่า "เงินในบัญชีติดลบได้ยังไง?!" แคมพอเราไปเปิดดู Log ใน Production สิ่งที่เราเห็นมีแค่คำว่า **Transfer started** กับ **Transfer success** เต็มไปหมด... หาต้นตอไม่ได้เลย

โจทย์: เปลี่ยนจากการ print() เป็นการใช้ logger โดยมีเงื่อนไข ดังนี้

1. warning เมื่อโอนเงินยอดติดลบ หรือ 0 บาท -> Transfer amount must be positive
2. error เมื่อยอดเงินคงเหลือไม่เพียงพอต่อยอดโอน -> Insufficient balance
3. info เมื่อโอนเงินสำเร็จ

และเขียน Unit Test ทดสอบตามฟังก์ชันในโค้ด tests.py

The Phantom Transfer Bug

```
python -m unittest part1_phantom_bug.tests
```


Log Detective

ไฟล์ JSON Log จำลองเหตุการณ์ระบบล่ม และให้ เขียน Python Script เพื่อสกัดข้อมูลและสร้างเงื่อนไขการแจ้งเตือน (Alert Logic) โดยมีเงื่อนไข ดังนี้

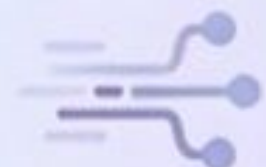
1. เขียนโค้ดอ่านไฟล์ JSON นี้ แล้วพิมพ์ออกมาเฉพาะ Log ที่มีระดับความรุนแรงเป็น ERROR เท่านั้น และนับว่ามี Error ทั้งหมดกี่ตัว
2. จาก Log ที่เป็น ERROR ทั้งหมด จงนับจัดกลุ่ม (Group by) ว่าพังจาก event อะไรมากที่สุด และถ้าเป็น checkout_failed ให้บอกด้วยว่า reason (เหตุผล) อะไรที่เกิดบ่อยสุด
3. เขียนฟังก์ชัน check_alert(logs) โดยมีเงื่อนไขว่า หากพบเหตุการณ์ checkout_failed ด้วยเหตุผล timeout เกิน 2 ครั้ง ภายในไฟล์นี้ ให้พิมพ์ข้อความว่า "CRITICAL ALERT: Payment Gateway อาจจะล่ม! พบ Timeout [จำนวน] ครั้ง"

Log Detective

python detective.py

WORKSHOP 02

การพัฒนา Chatbot เบื้องต้นสำหรับการรับแจ้งปัญหา



Web Chatbot (IT Support)

Software Development Life Cycle (SDLC) : User แจ้งปัญหาผ่าน Chatbot -> บอส่งข้อมูลลง Log (JSON) -> ระบบ Alert ทำงาน (Log Detective) -> Dev เข้าไปหา Root Cause -> Dev เขียน Test และแก้บั๊ก (Phantom Bug)

1. **Keyword Routing:** การดักจับคีย์เวิร์ดเพื่อเลือกคำตอบ
2. **State Management:** การจำสถานะของ User (ว่าคุยถึงขั้นตอนไหนแล้ว เช่น กำลังรอรับรหัสพนักงาน หรือ กำลังรอรับรายละเอียดปัญหา)
3. **Integration:** การสร้าง Ticket และคืนค่า Ticket ID (Correlation ID) ให้ User

แบบประเมินความพึงพอใจ



<https://forms.gle/U9HpgUcgxT1u3Q756>